PUBLIC

Code Assessment

of the Sulu Extensions XXII Smart Contracts

March 04, 2025

Produced for

enzyme

by

CHAINSECURITY

Contents

1 Executive Summary 3
2 Assessment Overview 5
3 Limitations and use of report 7
4 Terminology 8
5 Open Findings 9
6 Informational 10
7 Notes 11

1 Executive Summary

Dear Enzyme Team,

Thank you for trusting us to help Enzyme Foundation with this security audit. Our executive summary provides an overview of subjects covered in our audit of the latest reviewed contracts of Sulu Extensions XXII according to Scope to support you in forming an opinion on their security risks.

Enzyme Foundation implements a queue-based deposit mechanism for single asset deposits into the Enzyme vault. This system enables deposit requests to be queued and batched together later by an authorized user.

The most critical subjects covered in our audit are functional correctness, access control, and precision of arithmetic operations. Security regarding these subjects is high.

The general subjects covered are code complexity, documentation and specification, gas efficiency. Security regarding these subjects is high.

In summary, we find that the codebase provides a high level of security.

It is important to note that security audits are time-boxed and cannot uncover all vulnerabilities. They complement but don't replace other vital measures to secure a project.

The following sections will give an overview of the system, our methodology, the issues uncovered, and how they have been addressed. We are happy to receive questions and feedback to improve our service.

Sincerely yours,

ChainSecurity

1.1 Overview of the Findings

Below we provide a brief numerical overview of the findings and how they have been addressed.

Critical-Severity Findings	0
High-Severity Findings	0
Medium-Severity Findings	0
Low-Severity Findings	0

2 Assessment Overview

In this section, we briefly describe the overall structure and scope of the engagement, including the code commit which is referenced throughout this report.

2.1 Scope

The assessment was performed on the source code files inside the Sulu Extensions XXII repository based on the documentation files. The table below indicates the code versions relevant to this report and when they were received.

V	Date	Commit Hash	Note
1	28 February 2025	5c8a7793025ec2fd103fb5b2743bdfec64fa46bd	Initial Version

For the solidity smart contracts, the compiler version `0.8.19` was chosen.

The contracts in scope are the following:

```
contracts/persistent/single-asset-deposit-queue/SingleAssetDepositQueueLib.sol
contracts/persistent/single-asset-deposit-queue/ISingleAssetDepositQueue.sol
```

2.1.1 Excluded from scope

All contracts not explicitly in scope are out of scope. Third-party libraries such as OpenZeppelin are out of scope. The tokens that are handled by the contracts in scope are assumed not to exhibit unexpected behavior such as rebalancing and rebasing.

2.2 System Overview

This system overview describes the initially received version (Version 1) of the contracts as defined in the Assessment Overview.

Furthermore, in the findings section, we have added a version icon to each of the findings to increase the readability of the report.

Enzyme Foundation offers the `SingleAssetDepositQueueLib`, which implements a first-come first-served deposit queue mechanism for efficiently channeling assets into an Enzyme Vault. Funds will use AllowedDepositRecipientsPolicy with SingleAssetDepositQueue set as the only whitelisted address.

Deposit Queue Mechanism: The contract enables users to submit deposit requests via the `requestDeposit()` function. Each request is assigned a unique, sequential identifier and includes:

- A specified deposit asset amount that meets or exceeds a configurable minimum.
- A cancellation eligibility timestamp (determined by a minimum request time), before which the request cannot be canceled.

Queue Processing and Vault Integration: Once deposit requests are queued, designated managers or the vault owner can process them using the `depositFromQueue()` function. This function:

- Aggregates valid deposit requests within a defined range.
- Bypasses any requests explicitly flagged to be skipped.

- Transfers the total aggregated deposit amount to the vault via a pre-formatted call (leveraging the Enzyme Global Configuration). As of Version 1, only vaults deployed by `FUND_DEPLOYER_V4` can be deposited to. It is assumed that the DepositQueue is the only whitelisted depositor to the fund, as there is no slippage protection on this call.
- Receives vault shares in return, which are then dispersed pro-rata to each depositor according to their original contribution.

Request Cancellation: Users can cancel their deposit requests once the predefined cancellation time has elapsed, or if it has been bypassed. Only the original requester can initiate a cancellation, ensuring authorized control. When canceled, the request is removed from the queue and the deposited funds are refunded via token transfer.

2.2.1 Roles and Trust Model

We extend Sulu's trust model (see previous reports) as follows:

- **Depositors** initiate deposit requests and may cancel their own requests after the minimum request time elapses.
- **Managers** and the **Vault Owner** have elevated privileges, including the ability to process the deposit queue, adjust key parameters (e.g., minimum deposit amount, minimum request time, depositor allowlist), add or remove managers, and shut down the queue when necessary. The managers are trusted not to set a minimum request time that is too long, otherwise funds could be stuck indefinitely if they are not processed.
- The queue is upgradeable, as it's deployed behind a proxy. As a result, the proxy owner is fully trusted and has the power to seize all funds in the queue.

3 Limitations and use of report

Security assessments cannot uncover all existing vulnerabilities; even an assessment in which no vulnerabilities are found is not a guarantee of a secure system. However, code assessments enable the discovery of vulnerabilities that were overlooked during development and areas where additional security measures are necessary. In most cases, applications are either fully protected against a certain type of attack, or they are completely unprotected against it. Some of the issues may affect the entire application, while some lack protection only in certain areas. This is why we carry out a source code assessment aimed at determining all locations that need to be fixed. Within the customer-determined time frame, ChainSecurity has performed an assessment in order to discover as many vulnerabilities as possible.

The focus of our assessment was limited to the code parts defined in the engagement letter. We assessed whether the project follows the provided specifications. These assessments are based on the provided threat model and trust assumptions. We draw attention to the fact that due to inherent limitations in any software development process and software product, an inherent risk exists that even major failures or malfunctions can remain undetected. Further uncertainties exist in any software product or application used during the development, which itself cannot be free from any error or failures. These preconditions can have an impact on the system's code and/or functions and/or operation. We did not assess the underlying third-party infrastructure which adds further inherent risks as we rely on the correct execution of the included third-party technology stack itself. Report readers should also take into account that over the life cycle of any software, changes to the product itself or to the environment in which it is operated can have an impact leading to operational behaviors other than those initially determined in the business specification.

4 Terminology

For the purpose of this assessment, we adopt the following terminology. To classify the severity of our findings, we determine the likelihood and impact (according to the CVSS risk rating methodology).

- *Likelihood* represents the likelihood of a finding to be triggered or exploited in practice
- *Impact* specifies the technical and business-related consequences of a finding
- *Severity* is derived based on the likelihood and the impact

We categorize the findings into four distinct categories, depending on their severity. These severities are derived from the likelihood and the impact using the following table, following a standard risk assessment procedure.

Likelihood	Impact		
	High	Medium	Low
High	Critical	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

As seen in the table above, findings that have both a high likelihood and a high impact are classified as critical. Intuitively, such findings are likely to be triggered and cause significant disruption. Overall, the severity correlates with the associated risk. However, every finding's risk should always be closely checked, regardless of severity.

5 Open Findings

In this section, we describe our findings. The findings are split into these different categories:

Below we provide a numerical overview of the identified findings, split up by their severity.

Critical-Severity Findings	0
High-Severity Findings	0
Medium-Severity Findings	0
Low-Severity Findings	0

6 Informational

We utilize this section to point out informational findings that are less severe than issues. These informational issues allow us to point out more theoretical findings. Their explanation hopefully improves the overall understanding of the project's security. Furthermore, we point out findings which are unrelated to security.

6.1 Missing Indexes in Events

Informational **Version 1**

CS-SUL-22-001

`SingleAssetDepositQueueLib` defines a number of events such as `Deposited`, `RequestCanceled` which could use indexed parameters.

6.2 Redundant Event Emission

Informational **Version 1**

CS-SUL-22-002

`setMinDepositAssetAmount()`, `setMinRequestTime()`, and `setDepositorAllowlistId()` emit redundant events even when the respective values are not changed.

7 Notes

We leverage this section to highlight further findings that are not necessarily issues. The mentioned topics serve to clarify or support the report, but do not require an immediate modification inside the project. Instead, they should raise awareness in order to improve the overall understanding.

7.1 Share Price Must Not Become Inflated

Note **Version 1**

The shares returned to users in `depositFromQueue()` are calculated as follows:

```
userSharesAmountToDisperse = sharesBalanceToDisperse * depositAssetAmount / totalAssetsDeposited;
```

Due to the integer division, the shares returned to users may be rounded down. If the value of shares can be inflated so that a single wei of shares has a significant value, this could make it profitable to intentionally cause rounding for other users. The leftover funds would then be distributed the next time `depositFromQueue()` is called.

As a result, care should be taken that the share price cannot be inflated, e.g. through donation attacks. This is usually only possible when the number of shares in circulation is very low, e.g. when a fund is freshly deployed. Managers could monitor the share price and not process deposits if they detect abnormally high share prices.